

Software X: apsimNGpy Example

Magala et al. (2025)

Contents

1	Listing 1	1
2	Listing 2	1
3	Listing 3	2
4	Listing 4	3

1 Listing 1

```
from apsimNGpy.core.apsim import ApsimModel
from pathlib import Path
# Instantiate the model
model = ApsimModel(model='Maize', out_path='new_maize.apsimx')
# Edit the planting density in the manager node
model.edit_model(model_type='Models.Manager', model_name='Sow using a variable rule', Population=12)
# Download and replace weather data automatically
lonlat = (-93.44, 41.1234)
model.get_weather_from_web(lonlat=lonlat, start=1981, end=2022, source='daymet')
# Change the simulation start and end date
model.edit_model(model_type='Models.Clock', model_name='Clock',
                  start='1990-01-01', end='2021-12-31') # ISO 8601 format (YYYY-MM-DD)
# Run the model
model.run(report_name='Report')
# Get results
df = model.results
# Save the modified model
filename = str(Path(f"my_edited_maize_model_at_{lonlat}.apsimx").resolve())
model.save(file_name=filename)
```

2 Listing 2

```
from pathlib import Path
import os
from constants import custom_colors
from apsimNGpy.core.experimentmanager import ExperimentManager as Experiment
if __name__ == '__main__':
```

```

wd = Path(__file__).parent / 'apsimx'
wd.mkdir(exist_ok=True)
os.chdir(wd)
experiment = Experiment(model='Maize', out_path=(wd / 'out_maizer.apsimx').resolve())
experiment.init_experiment()
experiment.add_factor('[Fertilise at sowing].Script.Amount = 0, 300')
experiment.add_factor('[Sow using a variable rule].Script.CultivarName = Dekalb_XL82,
                      Melkassa, B_110, Pioneer_34K77')

# Run the experiment
experiment.run()
df = experiment.results
g = experiment.cat_plot(
    x='Amount', y='Yield', hue='CultivarName',
    kind='bar', height=8, aspect=1, palette=custom_colors,
)
# In Jupiter notebook, the graph is displayed even if show=False
experiment.render_plot(
    save_as=(wd / 'experiment.png').resolve(),
    dpi=600, show=True, ylabel='Simulated corn grain yield (kg ha−1)',
    xlabel='Nitrogen fertilizer (kg ha−1)',
)

```

3 Listing 3

```

from apsimNGpy.core.multicores import MultiCoreManager
from apsimNGpy.core.apsim import ApsimModel
from pathlib import Path
import numpy as np
if __name__ == "__main__":
    # Create dummy simulation jobs (replace with actual APSIM jobs)
    _jobs = set()
    for m in np.linspace(0, 300, 1000):
        model = ApsimModel("Maize", out_path = Path(f"__{m}__.apsimx").resolve())
        model.edit_model_by_path(".Simulations.Simulation.Field.Fertilise at sowing", Amount=m)
        _job.add(model.path)
    # Initialize the multi-core task manager
    task_manager = MultiCoreManager(db_path=Path("test.db").resolve(), agg_func=None)
    task_manager.run_all_jobs(_jobs, n_cores=10, threads=False, clear_db=True) #across 10 CPU cores
    # Retrieve and aggregate the simulation results
    df = task_manager.get_simulated_output(axis=0)
    # Equivalent to:
    data = task_manager.results

```

Console output (example)

```

Processing all jobs. Please wait!! :                               100.0% [1000/1000] Complete |
0.78s/iteration | Elapsed time: 00:12:55.093

```

there you go

4 Listing 4

```
from apsimNGpy.tests.unittests.test_factory import obs
from apsimNGpy.optimizer.problems.smp import MixedProblem
from apsimNGpy.optimizer.minimize.single_mixed import MixedVariableOptimizer
if __name__ == '__main__':
    # -----
    # Define everything below the module guard when using parallelism
    # -----

    mp = MixedProblem(model='Maize', trainer_dataset=obs, pred_col='Yield',
                      metric='rrmse', index='year', trainer_col='observed')
    soil_params = {
        "path": ".Simulations.Simulation.Field.Soil.Organic",
        "vtype": ['continuous(1, 500)', 'continuous(0.02, 0.06)'],
        "start_value": [100, 0.021],
        "candidate_param": ["FOM", 'FBiom'],
        "other_params": {"Carbon": 1.2},
    }
    cultivar_param = {
        "path": ".Simulations.Simulation.Field.Maize.CultivarFolder.Dekalb_XL82",
        "vtype": ['step_random_int(400, 600, q=5)', ],
        "start_value": [550, ],
        "candidate_param": ["[Grain].MaximumGrainsPerCob.FixedValue", ],
        "other_params": {"sowed": True},
        'cultivar': True # Signals to apsimNGpy to treat it as a cultivar parameter
    }

    mp.submit_factor(**soil_params)
    mp.submit_factor(**cultivar_param)
    minim = MixedVariableOptimizer(problem=mp)

    res = minim.minimize_with_de(use_threads=True, updating='deferred',
                                workers=10, popsize=20, constraints=(0, 0.2))

    # use a local solver
    minim.minimize_with_local(method='Nelder-Mead') # constraint free
    # over 14 bounded and non bounded local solver algorithms are available
```

listing 1 a

a
a
a
a
a a
a
a
a
a
a
a

a a

a

a

```
# Pinning workflow to a specific APSIM version or bin_path
from apsimNGpy.core.config import apsim_bin_context
with apsim_bin_context(dotenv_path='./config_env/.env', bin_key='BIN'):
    from apsimNGpy.core.apsim import ApsimModel # uses the bin path in .env file for loading
if __name__ == '__main__':
    # use a context manager to delete temporal files automatically
    with ApsimModel('Maize') as model:
        manager_scripts = model.inspect_model('Models.Manager', fullpath=True)
        print(manager_scripts)
        ['.Simulations.Simulation.Field.Sow using a variable rule',
         '.Simulations.Simulation.Field.Fertilise at sowing', '.Simulations.Simulation.Field.Harvest']
        # Get the names of the manager scripts only
        manager_scripts = model.inspect_model('Models.Manager', fullpath=False)
        print(manager_scripts)
        ['Sow using a variable rule', 'Fertilise at sowing', 'Harvest']
        # Inspect the parameters for sow using a variable rule
        # we have the path from above
        sowp = model.inspect_model_parameters_by_path(
            '.Simulations.Simulation.Field.Sow using a variable rule', )
        print(sowp)
        {'Crop': '[Maize]',
         'StartDate': '1-nov',
         'EndDate': '10-jan',
         'MinESW': '100',
         'MinRain': '25',
         'RainDays': '7',
         'CultivarName': 'Dekalb_XL82',
         'SowingDepth': '30',
         'RowSpacing': '750',
         'Population': '12'}
        # inspect the simulation structure and see the relationship between models
        model.inspect_file(cultivar=False) # cultivar specific node are so many turn them off
        # if that is not enough, open the file in the GUI

        model.preview_simulation(
            watch=True) # watch is a Boolean that makes it possible to stream live the changes.
        #Please close the GUI when completed before proceeding back
        # Press `Ctrl+C` in the corresponding cell to stop watching.
```